

CNN Architectures: From LeNet-5 to EfficientNet-V2

Praveen Kumar Chandaliya

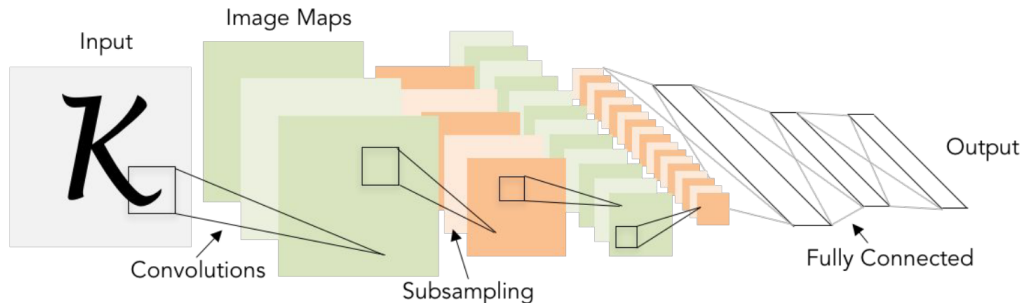
Sardar Vallabhabhi National Institute of Technology, Surat

February 4, 2026

<https://acesse.one/hWoww>

LeNet5

[LeCun et al., 1998]

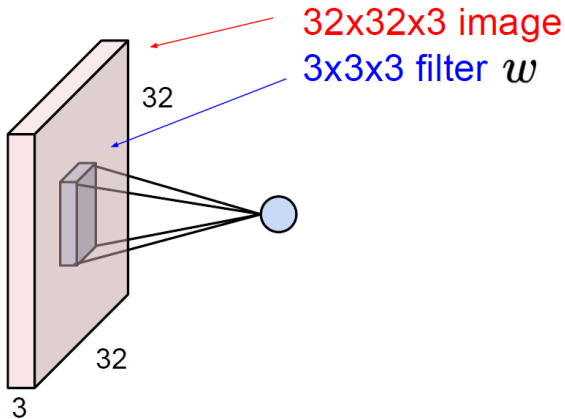


Conv filters were 5x5, applied at stride 1
Subsampling (Pooling) layers were 2x2 applied at stride 2
i.e. architecture is [CONV-POOL-CONV-POOL-FC-FC]

1

¹Fei-Fei Li, Ranjay Krishna, Danfei Xu, CS213n Deep Learning for computer vision, Lecture 9

Review: Convolution



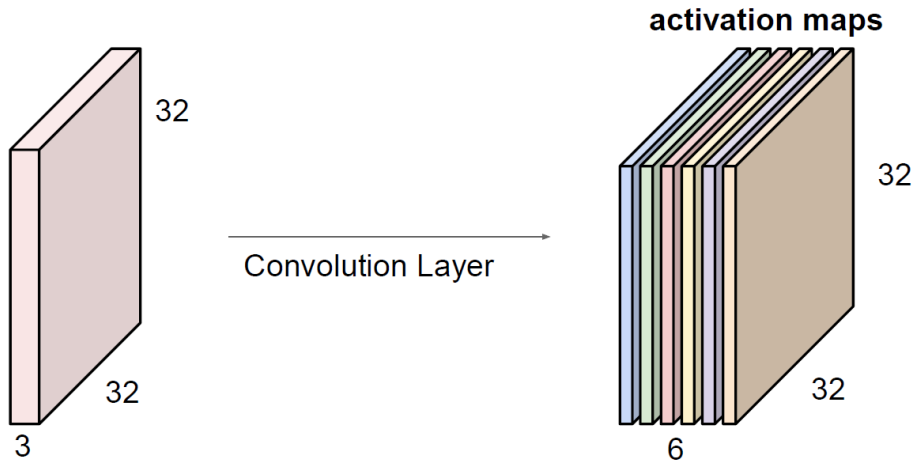
0	0	0	0	0	0				
0	0	0	0	0	0				
0	0	0	0	0	0				
0									
0									

Stride:
Downsample
output activations

0	0	0	0	0	0	0			
0	0	0	0	0	0	0			
0	0	0	0	0	0	0			
0									
0									

Padding:
Preserve
input spatial
dimensions in
output activations

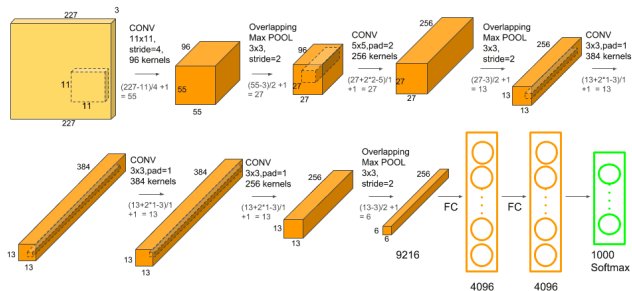
Review: Convolution



CNN Architectures

- AlexNet
- VGG
- GoogLeNet (Inception)
- ResNet
- DenseNet
- SEnet
- Wide ResNet
- ResNeXt
- MobileNet
- NASNet
- EfficientNet

AlexNet



2

AlexNet takes a 227×227 RGB image as input and consists of five convolutional layers followed by three fully connected layers. The network employs ReLU activations, overlapping max pooling, local response normalization, and dropout to accelerate training and mitigate overfitting. Trained using GPUs.

²<https://learnopencv.com/understanding-alexnet/>

AlexNet Parameters

Size / Operation	Filter	Depth	Stride	Padding	Number of Parameter
3* 227 * 227					
Conv1	11 * 11	96	4		$(11*11*3 + 1) * 96=34944$
96 * 55 * 55					
Max Pooling	3 * 3		2		
96 * 27 * 27					
Conv2	5 * 5	256	1	2	$(5 * 5 * 96 + 1) * 256=614656$
256 * 27 * 27					
Max Pooling	3 * 3		2		
256 * 13 * 13					
Conv3	3 * 3	384	1	1	$(3 * 3 * 256 + 1) * 384=885120$
384 * 13 * 13					
Conv4	3 * 3	384	1	1	$(3 * 3 * 384 + 1) * 384=1327488$
384 * 13 * 13					
Conv5	3 * 3	256	1	1	$(3 * 3 * 384 + 1) * 256=884992$
256 * 13 * 13					
Max Pooling	3 * 3		2		
256 * 6 * 6					
FC6					$256 * 6 * 6 * 4096=37748736$
4096					
FC7					$4096 * 4096=16777216$
4096					
FC8					$4096 * 1000=4096000$
1000 classes					
Overall					62369152=62.3 million
Conv VS FC	Conv:3.7million (6%) , FC: 58.6 million (94%)				

AlexNet

Key Idea

First deep CNN to win ImageNet, proving deep learning dominance.

- 8 layers: 5 convolution + 3 fully connected
- Introduced ReLU for faster training
- Used Norm layers (not common anymore)
- heavy data augmentation
- Used Dropout to reduce overfitting (0.5)
- SGD Momentum 0.9
- Learning rate $1e-2$, reduced by 10 manually when val accuracy plateaus
- L2 weight decay $5e-4$
- Trained on GPUs (Trained on GTX 580 GPU with only 3 GB of memory. Network spread across 2 GPUs, half the neurons (feature maps) on each GPU)

Limitation

Very large number of parameters ($\sim 60M$)

What is ImageNet?

3

- **Core Concept:** Based on the **WordNet** hierarchy.
- **Scale:** Over 14 million manually annotated images.
- **Impact:** The primary benchmark for the "Deep Learning Revolution" since 2012.

Metric	Full ImageNet (21K)	ImageNet-1K
Total Images	~14.2 Million	1,281,167 (Train)
Total Classes	21,841	1,000
Hierarchy	27 High-level categories	Flat 1,000 classes
Bounding Boxes	~1.03 Million	~150,000+

Table 1: Comparison of the Full vs. ILSVRC subsets.

³<https://image-net.org/>

ImageNet-1K: The Research Standard

Training Split

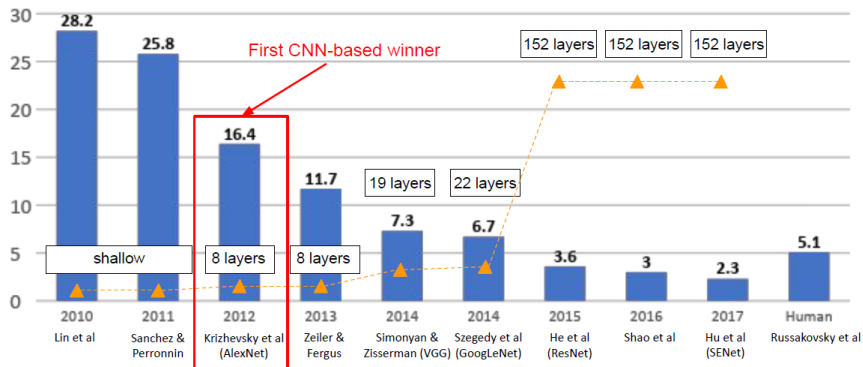
1,281,167 images used to train model weights.

Validation Split

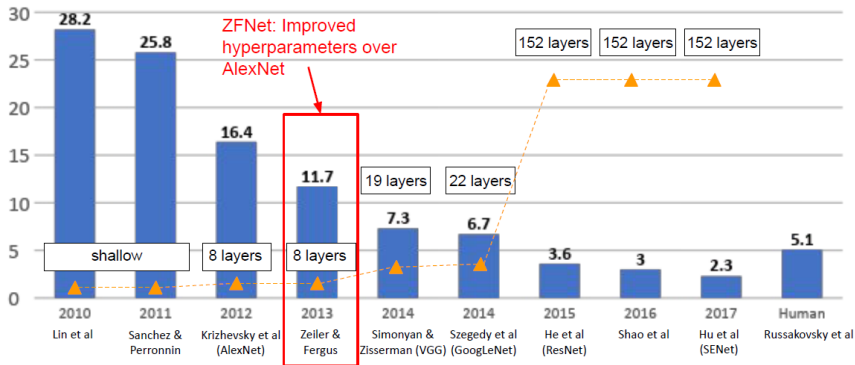
50,000 images (exactly 50 per class) used for hyperparameter tuning.

- Labels for the test set (100k images) remain private.
- Standard resolution: 224×224 or 256×256 pixels.

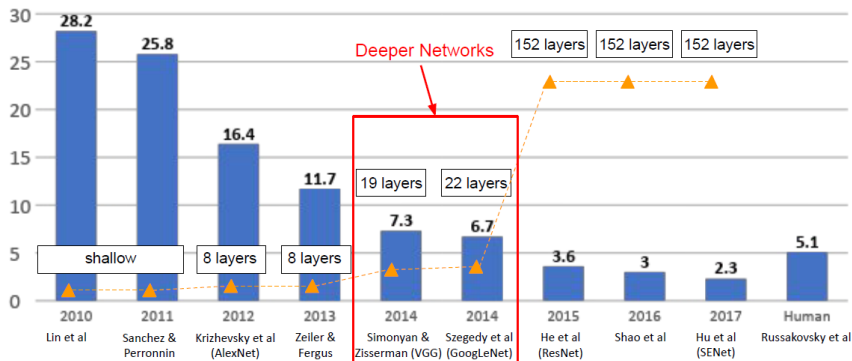
ImageNet Large Scale Visual Recognition Challenge



ImageNet Large Scale Visual Recognition Challenge



ImageNet Large Scale Visual Recognition Challenge



Performance Benchmarks: 2025

- **Top-1 Accuracy:** Elite models (ViT-G, MaX-ViT) reach **90–91%**.
- **Top-5 Accuracy:** Now frequently exceeds **98%**.
- **Pre-training Trend:** Modern models pre-train on ImageNet-21K and fine-tune on ImageNet-1K.

"ImageNet-1K accuracy is now used more for measuring efficiency rather than just raw capacity."

VGG

Small filters, Deeper networks

8 layers (AlexNet)

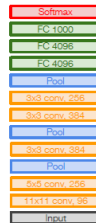
-> 16 - 19 layers (VGG16Net)

Only 3x3 CONV stride 1, pad 1
and 2x2 MAX POOL stride 2

11.7% top 5 error in ILSVRC'13 (ZFNet)

-> 7.3% top 5 error in ILSVRC'14

[Simonyan and Zisserman, 2014]



AlexNet



VGG16

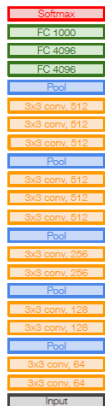
VGG19

VGG FLOPS and Parameters

INPUT: [224x224x3] memory: $224*224*3=150K$ params: 0 (not counting biases)
 CONV3-64: [224x224x64] memory: $224*224*64=3.2M$ params: $(3*3*3)*64 = 1,728$
 CONV3-64: [224x224x64] memory: $224*224*64=3.2M$ params: $(3*3*64)*64 = 36,864$
 POOL2: [112x112x64] memory: $112*112*64=800K$ params: 0
 CONV3-128: [112x112x128] memory: $112*112*128=1.6M$ params: $(3*3*64)*128 = 73,728$
 CONV3-128: [112x112x128] memory: $112*112*128=1.6M$ params: $(3*3*128)*128 = 147,456$
 POOL2: [56x56x128] memory: $56*56*128=400K$ params: 0
 CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*128)*256 = 294,912$
 CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*256)*256 = 589,824$
 CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*256)*256 = 589,824$
 POOL2: [28x28x256] memory: $28*28*256=200K$ params: 0
 CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*256)*512 = 1,179,648$
 CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*512)*512 = 2,359,296$
 CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*512)*512 = 2,359,296$
 POOL2: [14x14x512] memory: $14*14*512=100K$ params: 0
 CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$
 CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$
 CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$
 POOL2: [7x7x512] memory: $7*7*512=25K$ params: 0
 FC: [1x1x4096] memory: 4096 params: $7*7*512*4096 = 102,760,448$
 FC: [1x1x4096] memory: 4096 params: $4096*4096 = 16,777,216$
 FC: [1x1x1000] memory: 1000 params: $4096*1000 = 4,096,000$

TOTAL memory: 24M * 4 bytes $\approx$ 96MB / image (for a forward pass)

TOTAL params: 138M parameters



VGG16

VGG

Key Idea

Depth through simplicity: small filters, deep networks.

- Uses only 3×3 convolution filters
- VGG-16 and VGG-19 are popular variants
- Uniform architecture throughout
- ILSVRC'14 2nd in classification, 1st in localization
- Similar training procedure as Krizhevsky 2012 (AlexNet)
- Use VGG16 or VGG19 (VGG19 only slightly better, more memory)
- Use ensembles for best results
- FC7 features generalize well to other tasks

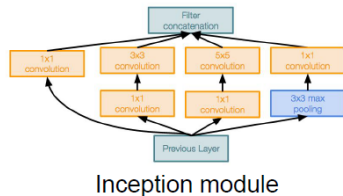
Limitation

Very high memory and computation cost

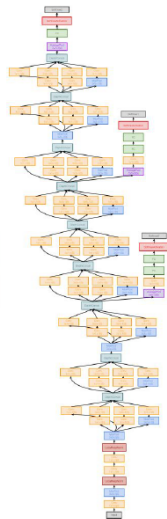
GoogLeNet (Inception):2014

Deeper networks, with computational efficiency

- ILSVRC'14 classification winner (6.7% top 5 error)
- 22 layers
- Only 5 million parameters!
12x less than AlexNet
27x less than VGG-16
- Efficient "Inception" module
- No FC layers

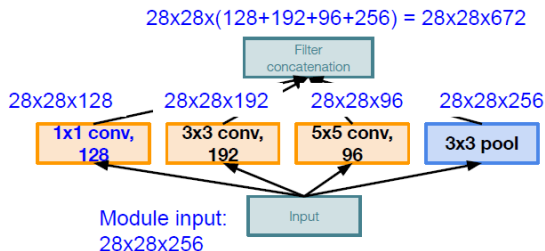


[Szegedy et al., 2014]



GoogLeNet (Inception)

What is output size after filter concatenation?



Naive Inception module
[Szegedy et al., 2014]

What is the problem with this?
[Hint: Computational complexity]

Conv Ops:

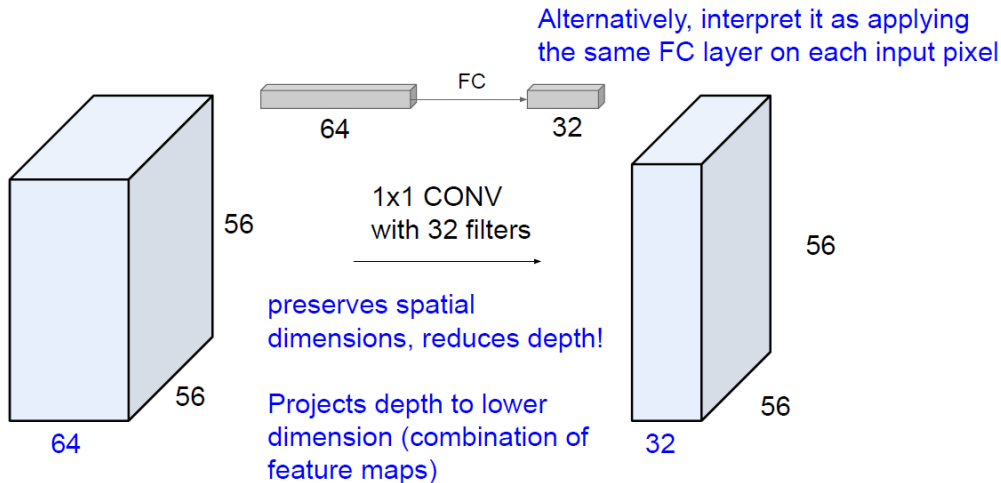
[1×1 conv, 128] $28 \times 28 \times 128 \times 1 \times 1 \times 256$
 [3×3 conv, 192] $28 \times 28 \times 192 \times 3 \times 3 \times 256$
 [5×5 conv, 96] $28 \times 28 \times 96 \times 5 \times 5 \times 256$

Total: 854M ops

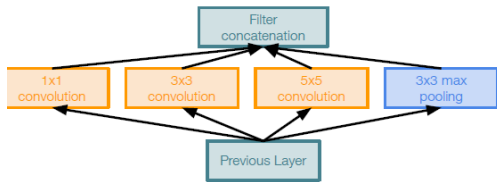
Very expensive compute

Pooling layer also preserves feature depth, which means total depth after concatenation can only grow at every layer!

1x1 Convolution

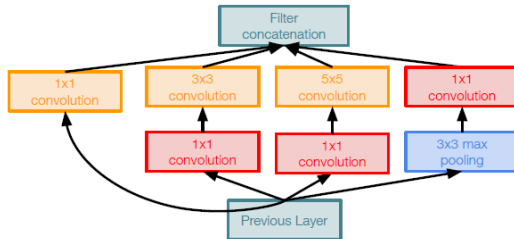


Naive Inception to dimension reduction



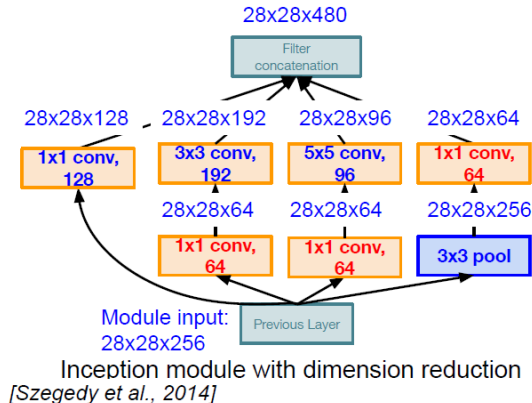
Naive Inception module

1x1 conv "bottleneck"
layers



Inception module with dimension reduction

GoogLeNet



Using same parallel layers as naive example, and adding “1x1 conv, 64 filter” bottlenecks:

Conv Ops:

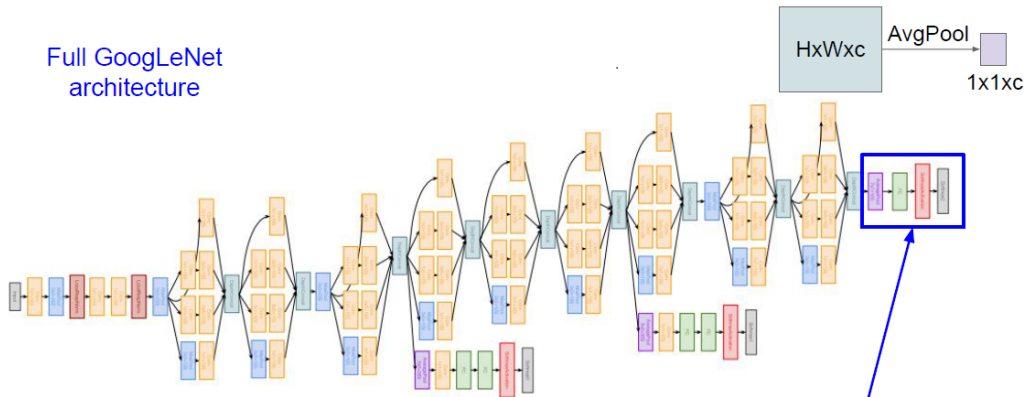
[1x1 conv, 64] 28x28x64x1x1x256
 [1x1 conv, 64] 28x28x64x1x1x256
 [1x1 conv, 128] 28x28x128x1x1x256
 [3x3 conv, 192] 28x28x192x3x3x64
 [5x5 conv, 96] 28x28x96x5x5x64
 [1x1 conv, 64] 28x28x64x1x1x256

Total: 358M ops

Compared to 854M ops for naive version
 Bottleneck can also reduce depth after pooling layer

GoogLeNet

Full GoogLeNet
architecture



Note: after the last convolutional layer, a global average pooling layer is used that spatially averages across each feature map, before final FC layer. No longer multiple expensive FC layers!

Classifier output

[Szegedy et al., 2014]

GoogLeNet

Key Idea

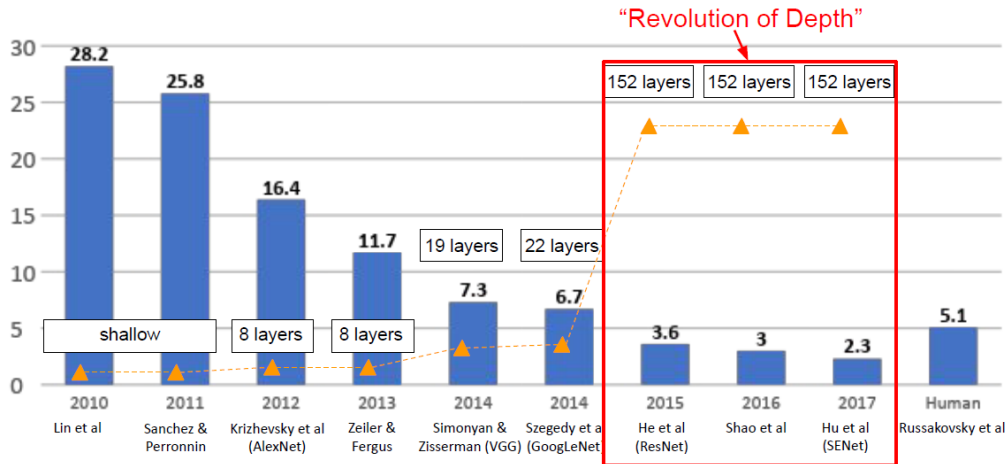
Multi-scale feature extraction in the same layer.

- Apply parallel filter operations on the input from previous layer (Inception module): Multiple receptive field sizes for convolution 1×1 , 3×3 , 5×5 convolutions
- Pooling operation (3×3)
- Concatenate all filter outputs together channel-wise
- Uses 1×1 convolution for dimensionality reduction
- “Inception module”: design a good local network topology (network within a network) and then stack these modules on top of each other
- Much fewer parameters than VGG

Innovation

Efficient depth without computational explosion

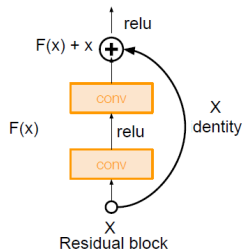
Revolution of Depth



ResNet

Very deep networks using residual connections

- 152-layer model for ImageNet
- ILSVRC'15 classification winner (3.57% top 5 error)
- Swept all classification and detection competitions in ILSVRC'15 and COCO'15!



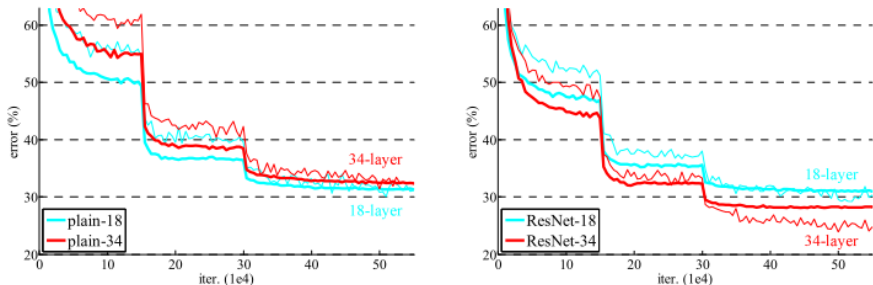
[He et al., 2015]



Residual Block

To fix the gradient flow, group layers into blocks and introduce a new path (Skip Connection). Data flows through the block and around the block simultaneously.

Plain vs Residual

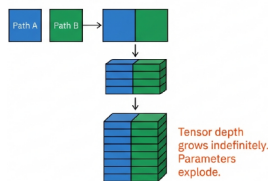


Training on **ImageNet**. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts.

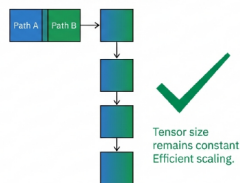
- The plain 34-layer network shows a higher training error than the 18-layer model (Figure, left), illustrating the degradation problem.
- In contrast, the 34-layer ResNet achieves a lower training error than the 18-layer model (Figure, right), demonstrating that residual learning mitigates degradation and enables accuracy gains with increased depth.

ResNet: Concatenation and Element-wise Addition

Concatenation

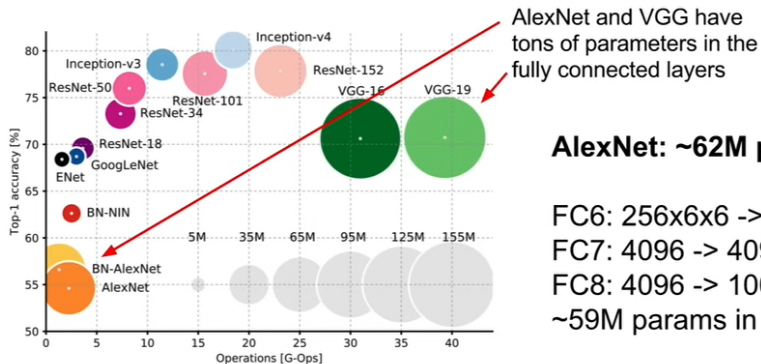


Element-wise Addition



- Learning residuals instead of direct mappings.
- Skip connections: $y = F(x) + x$
- Enables very deep networks (up to 152 layers)
- Solves vanishing gradient problem.
- ResNet-18, ResNet-50, ResNet-101
- Foundation for most modern CNNs.

CNN Parameter



AlexNet: ~62M parameters

FC6: 256x6x6 -> 4096: 38M params

FC7: 4096 -> 4096: 17M params

FC8: 4096 -> 1000: 4M params

~59M params in FC layers!

DenseNet:2017

Key Idea

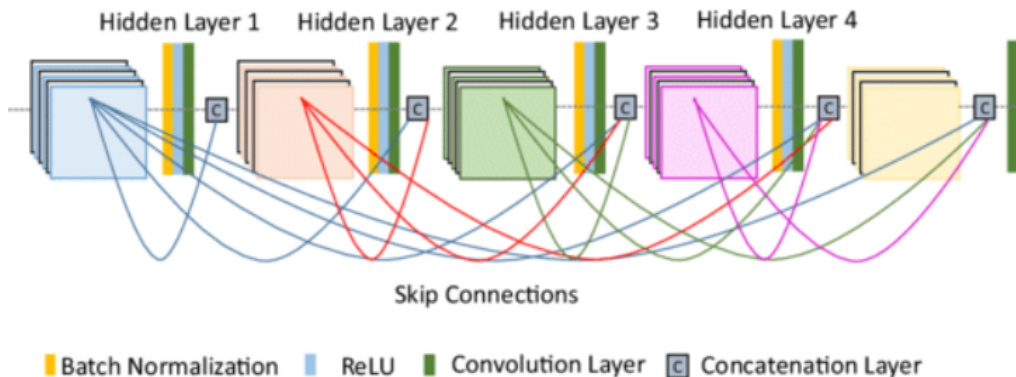
DenseNet, short for Densely Connected Convolutional Network. It revolutionized CNNs by introducing dense connectivity, where each layer is connected to every other layer in a feed-forward manner. This design improves gradient flow, feature reuse, and parameter efficiency, making DenseNet highly effective for tasks like image classification, object detection, and semantic segmentation. Each layer connects to all previous layers.

- Feature reuse improves efficiency
- Strong gradient flow
- Fewer parameters than ResNet
- every layer has direct access to gradients and input from the earlier layers.
- fewer parameter that traditional networks

Trade-off

High memory usage due to dense connections

DenseNet121



ResNet vs DenseNet

ResNet improves deep network optimization using additive skip connections, while DenseNet enhances feature reuse and gradient flow through dense feature concatenation, often achieving similar accuracy with fewer parameters.

Squeeze-and-Excitation Networks:2019

Key Idea

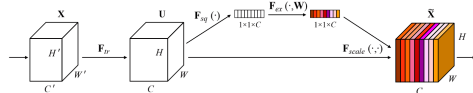
Channel-wise attention mechanism.

- Learns importance of feature channels
- Squeeze: global average pooling to capture global context.
- Excitation: adaptively recalibrate channel responses (channel re-weighting), allowing the network to emphasize informative features and suppress less useful ones.

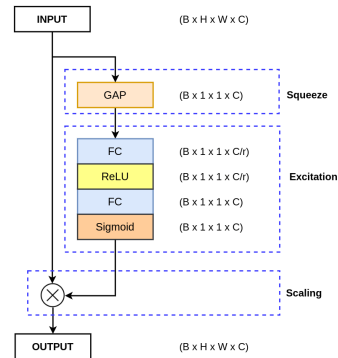
Advantage

- Improves performance with minimal overhead by emphasizes informative channels and suppresses weak ones.
- ResNet adds features, DenseNet concatenates features, and SE Networks reweight features using learned channel attention.

Squeeze-and-Excitation Networks



(a) SE Block – Squeeze Operation



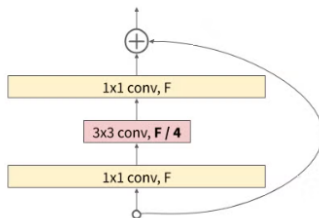
(b) SE Block – Excitation Operation

Figure 1: Architecture of the Squeeze-and-Excitation (SE) Block

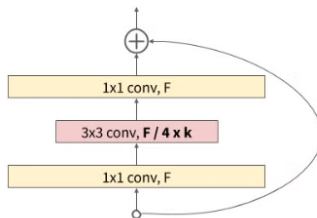
Wide ResNet

Wide ResNet improves performance not by increasing depth, but by increasing the width (number of channels) of residual blocks, addressing diminishing returns and optimization difficulties in very deep ResNets.

- Fewer layers, more feature maps
- Faster training than deep ResNet
- Better performance in practice



ResNet bottleneck



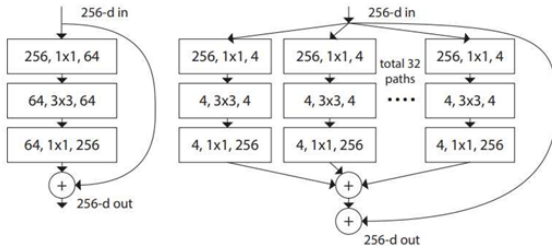
Wide ResNet bottleneck

ResNeXt

Key Idea

ResNeXt improves ResNet by introducing cardinality multiple parallel transformations aggregated together.

- Split-transform-merge strategy
- Depth $\rightarrow$ number of layers, Width $\rightarrow$ number of channels, Cardinality $\rightarrow$ number of parallel paths
- Improves accuracy without increasing complexity



MobileNet

Key Idea

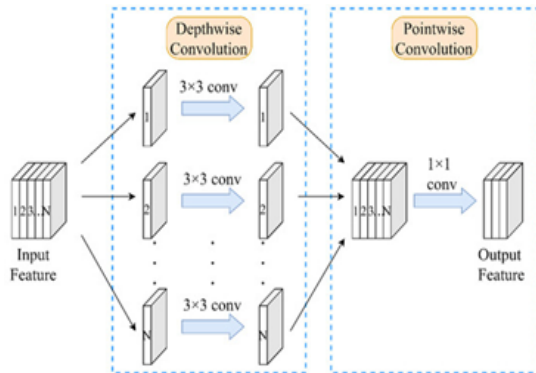
Efficient CNNs for mobile and edge devices.

- Depthwise separable convolutions
- Reduces computation by $\sim 9\times$
- Used in real-time applications
- MobileNetV1 (Depthwise separable convolution), MobileNetV2 (Inverted residuals + linear bottleneck), MobileNetV3 (NAS + SE attention + h-swish)

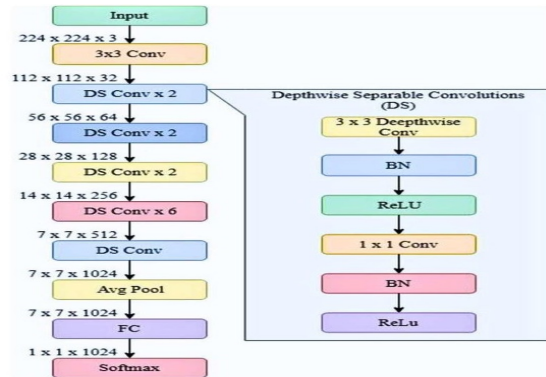
Use Case

Mobile, IoT, embedded AI

MobileNetV1



(a) Depth wise and pointwise convolution



(b) MobileNetV1

EfficientNet

Problem:

Traditional model scale arbitrarily by: Adding depth (more layers), Adding width (more channels), Increasing the image resolution. But this scaling is unbalanced and inefficient.

Key Idea

EfficientNet is a family of convolutional neural networks that uniformly scale network depth, width, and input resolution using a compound coefficient

- Developed by Google (2019)
- Based on a baseline model: EfficientNet-B0
- Balanced scaling strategy
- EfficientNet-B0 to B7
- Excellent accuracy–efficiency tradeoff

Why Compound Scaling?

- Scaling CNNs by only **depth**, **width**, or **resolution** is inefficient
- Over-scaling one dimension creates optimization or representation bottlenecks
- **Compound scaling** scales all dimensions jointly and systematically

Compound Scaling Principle

Given a baseline network with depth d , width w , and resolution r :

$$d(\phi) = \alpha^\phi \cdot d$$

$$w(\phi) = \beta^\phi \cdot w$$

$$r(\phi) = \gamma^\phi \cdot r$$

$$\text{subject to: } \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

- ϕ controls overall model scale
- α, β, γ balance depth, width, and resolution

Numerical Example: Baseline

Baseline network:

- Depth: $d = 10$
- Width: $w = 32$
- Resolution: $r = 224$

Scaling coefficients:

$$\alpha = 1.2, \quad \beta = 1.1, \quad \gamma = 1.15$$

Constraint check:

$$\alpha\beta^2\gamma^2 \approx 1.93 \approx 2$$

Compound Scaling: $\phi = 1$

$$d(1) = 1.2 \times 10 = 12$$

$$w(1) = 1.1 \times 32 \approx 35$$

$$r(1) = 1.15 \times 224 \approx 258$$

- All dimensions grow proportionally
- No single bottleneck dominates

Compound Scaling: $\phi = 2$

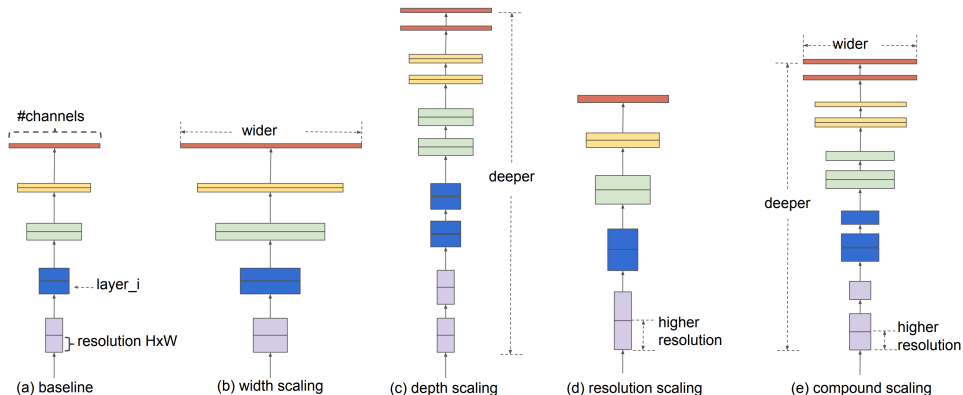
$$d(2) \approx 14$$

$$w(2) \approx 39$$

$$r(2) \approx 296$$

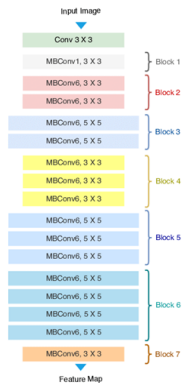
- $\sim 4\times$ computation vs baseline
- Balanced capacity increase

EfficientNet Compound Scaling

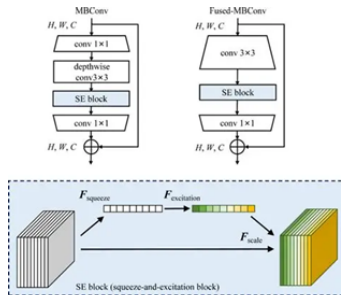


Model Scaling. (a) is a baseline network example; (b)-(d) are conventional scaling that only increases one dimension of network width, depth, or resolution. (e) is our proposed compound scaling method that uniformly scales all three dimensions with a fixed ratio.

EfficientNet



(a) EfficientNet



(b) Mobile Inverted Bottleneck blocks + Squeeze-and-Excitation

Stage i	Operator $\hat{\mathcal{F}}_i$	Resolution $\hat{H}_i \times \hat{W}_i$	#Channels $\hat{C}_i$	#Layers $\hat{L}_i$
1	Conv3x3	224×224	32	1
2	MBConv1, k3x3	112×112	16	1
3	MBConv6, k3x3	112×112	24	2
4	MBConv6, k5x5	56×56	40	2
5	MBConv6, k3x3	28×28	80	3
6	MBConv6, k5x5	14×14	112	3
7	MBConv6, k5x5	14×14	192	4
8	MBConv6, k3x3	7×7	320	1
9	Conv1x1 & Pooling & FC	7×7	1280	1

(c) Efficient B0 -B7

EfficientNet Architecture

Compound Scaling

Scaling depth, width, and resolution together leads to:

- Better accuracy–efficiency trade-off
- Predictable computational growth
- More effective use of model capacity

EfficientNet Network Component

- Built using **MBConv** blocks
- Depthwise separable convolutions
- Squeeze-and-Excitation (SE) blocks
- Swish (SiLU) activation

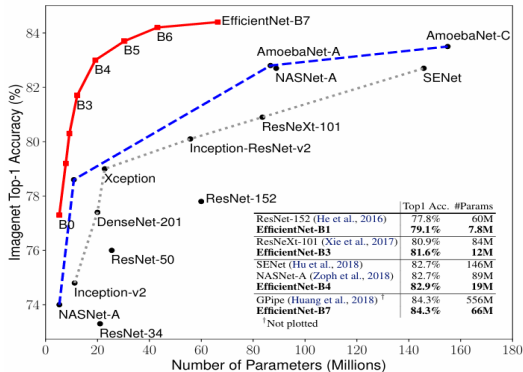
EfficientNet Variants (B0–B7)

Model	Resolution	Params	FLOPs
B0	224×224	5.3M	0.39B
B1	240×240	7.8M	0.70B
B2	260×260	9.2M	1.0B
B3	300×300	12M	1.8B
B4	380×380	19M	4.2B
B5	456×456	30M	9.9B
B6	528×528	43M	19B
B7	600×600	66M	37B

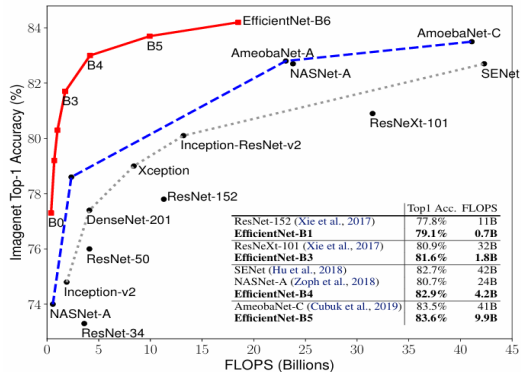
When to Use Which Variant?

- **B0–B1**: Mobile and edge devices
- **B2–B3**: Academic experiments and coursework
- **B4–B5**: High-accuracy vision tasks
- **B6–B7**: Benchmarking and large-scale systems

Model Size vs. ImageNet Accuracy



Model Size vs. ImageNet Accuracy



FLOPS vs. ImageNet Accuracy

EfficientNets significantly outperform other ConvNets. In particular, EfficientNet-B7 achieves new state-of-the-art 84.3% top-1 accuracy but being 8.4x smaller and 6.1x faster than GPipe.

EfficientNet-B1 is 7.6x smaller and 5.7x faster than ResNet-152

Neural Architecture Search

Motivation

- Neural network design is manual and expertise-driven
- Large design space: layers, operations, connections
- Trial-and-error is expensive and inefficient
- NAS automates architecture design

Definition

Neural Architecture Search (NAS) is an automated approach to discover optimal neural network architectures using algorithmic search strategies.

- Reduces human bias
- Often outperforms handcrafted architectures

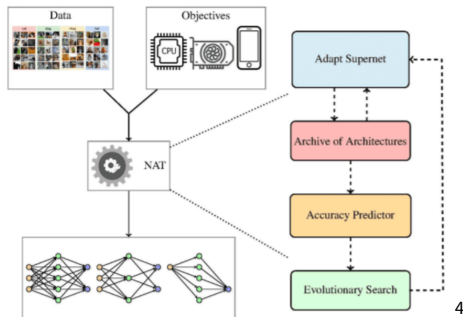
Core Components of NAS

- ① **Search Space** – possible architectures
- ② **Search Strategy** – how to explore the space
- ③ **Performance Estimation** – how to evaluate architectures

Main Challenge

Balancing search accuracy and computational cost

NAS Workflow



Typical Neural Architecture Search pipeline

- Controller proposes architecture
- Model is trained and evaluated
- Feedback guides the next search step

⁴NAS <https://theaisummer.com/neural-architecture-search/>

Types of NAS Methods

- **Reinforcement Learning-based NAS**
- **Evolutionary Algorithms**
- **Gradient-based NAS (DARTS)**
- **One-Shot NAS (weight sharing)**

DARTS: Gradient-Based NAS

- DARTS = Differentiable Architecture Search
- Makes architecture selection differentiable
- Optimizes weights and architecture jointly
- Extremely fast compared to RL-based NAS

Key Advantage

Reduces NAS cost from thousands of GPU-days to a few GPU-hours

EfficientNet: NAS in Practice

- EfficientNet uses NAS to design baseline architecture (B0)
- Applies compound scaling:
Depth, Width, Resolution
- Produces variants B0–B7

Impact

State-of-the-art accuracy with fewer parameters and FLOPs

Summary

- NAS automates neural network architecture design
- Consists of search space, strategy, and evaluation
- DARTS enables fast, differentiable NAS
- EfficientNet demonstrates NAS success in real-world models

EfficientNet (Version 1)

- EfficientNet introduced a principled approach to CNN scaling
- Uses **compound scaling** of:
 - Network depth
 - Network width
 - Input resolution
- Architecture based on MBConv blocks
- Achieved strong accuracy with fewer parameters
- Optimized primarily for accuracy–efficiency tradeoff

Limitation

Training can be slow for large-scale models

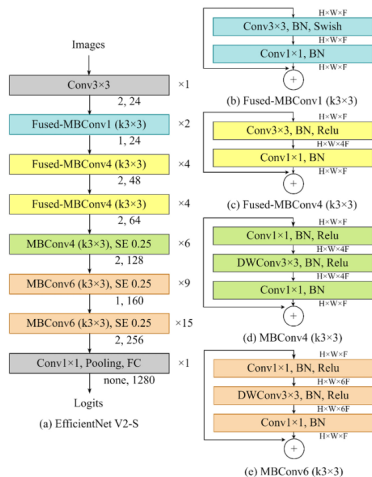
Motivation for EfficientNetV2: Smaller Models and Faster Training

- Growing demand for faster model training
- Deployment on diverse hardware platforms
- Need to reduce parameter count without accuracy loss
- Vision Transformers highlighted efficiency challenges in CNNs

EfficientNetV2: Key Innovations

- Introduces **training-aware Neural Architecture Search**
- Explicitly optimizes architectures for:
 - Training speed
 - Hardware efficiency
- Reduces overall parameter count

EfficientNetV2 Overview



- **Fused MBConv layers** in early network stages
- Reduces memory access cost
- Improves hardware utilization
- **Progressive learning strategy:**
 - Dynamically adjusts regularization
 - Adapts to increasing image resolution
- Fused-MBConv used in early stages
- MBConv retained in deeper layers
- Balances speed and parameter efficiency
- Faster training compared to MBConv

What is Fused-MBConv?

Why Fused-MBConv

- MBConv is very efficient for inference and Early layers have large spatial resolution
- Depthwise + pointwise convolutions are slow on GPUs/TPUs during training
- Fused-MBConv trades a bit of parameter efficiency for much faster training

Definition

Fused-MBConv (Fused Mobile Inverted Bottleneck Convolution) is a modified bottleneck block that combines expansion and spatial convolution into a single regular convolution.

- 1 Introduced in EfficientNetV2 and Optimized for hardware efficiency
- 2 **Fused $k \times k$ Standard Convolution**
 - Performs channel expansion and spatial filtering together
- 3 **Squeeze-and-Excitation (optional)**
- 4 **1×1 Projection Convolution**
- 5 **Skip Connection** (if stride = 1 and channels match)

Summary of Operations

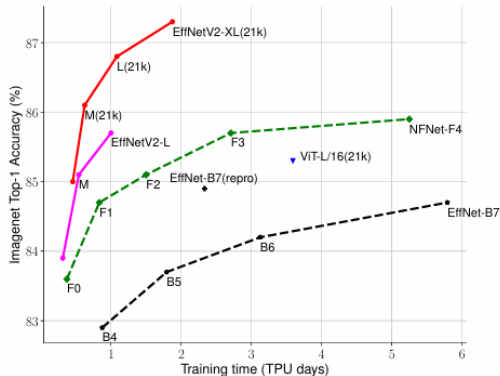
Table 2: Summary of Operations in a Fused-MBConv Block

Stage	Operation
1	Input feature map
2	$k \times k$ Convolution (Expansion + Spatial fused)
3	Batch Normalization
4	SiLU / Swish activation
5	1×1 Projection Convolution
6	Batch Normalization
7	Skip connection (if applicable)

Fused-MBConv vs MBConv

Aspect	MBConv	Fused-MBConv
Expansion	1×1 Conv	Included in $k \times k$ Conv
Spatial Conv	Depthwise	Standard Conv
Parameters	Lower	Slightly Higher
Memory Access	High	Lower
Training Speed	Slower	Faster
Usage	Deeper layers	Early layers

Performance Advantages



- Faster training compared to EfficientNetV1
- Improved inference latency
- Outperforms:
 - Previous EfficientNet variants
 - Modern Vision Transformers
- Strong results across multiple image classification benchmarks

Conclusion

- EfficientNetV2 balances architecture design and training strategy
- Training-aware NAS is critical for modern deep models
- Smarter training protocols lead to state-of-the-art performance
- Demonstrates CNN competitiveness against Vision Transformers

CNN Architecture Comparison

- **AlexNet**: First deep CNN
- **VGG**: Depth via simplicity
- **GoogLeNet**: Multi-scale features
- **ResNet**: Skip connections
- **DenseNet**: Feature reuse
- **SENet**: Channel attention
- **MobileNet**: Lightweight models
- **EfficientNet**: Optimal scaling
- **EfficientNetV2**: Faster training compared to EfficientNetV1
- Resources:
 - <https://www.youtube.com/watch?v=Wb61KRyNViI>
 - <https://www.youtube.com/watch?v=DiL3x2ctcm0>
 - <https://www.youtube.com/watch?v=CTsSrOKSPNo>

CNN Architecture Comparison

Model	Params (M)	FLOPs (G)	Top-1 Acc. (%)
AlexNet	60	0.7	57.2
VGG-16	138	15.5	71.5
GoogLeNet	6.8	1.6	74.8
ResNet-50	25.6	4.1	76.0
DenseNet-121	8.0	2.9	74.9
SENet-154	115	20.7	81.3
Wide ResNet	68	11.4	78.5
ResNeXt-50	25	4.2	77.8
MobileNetV2	3.4	0.3	71.8
NASNet-A	88.9	23.8	82.7
EfficientNet-B7	66	37.0	84.4

Table 3: Comparison on ImageNet

CNN Architecture Comparison

- **AlexNet**: First deep CNN
- **VGG**: Depth via simplicity
- **GoogLeNet**: Multi-scale features
- **ResNet**: Skip connections
- **DenseNet**: Feature reuse
- **SENet**: Channel attention
- **MobileNet**: Lightweight models
- **EfficientNet**: Optimal scaling

Vanishing Gradient Problem

- Deep neural networks require effective gradient flow
- During training, gradients are propagated using backpropagation
- In very deep networks, gradients may become extremely small
- This leads to the **Vanishing Gradient Problem**

Definition

Vanishing Gradient occurs when gradients shrink exponentially as they propagate backwards through deep networks, causing early layers to learn very slowly.

- Mostly affects early layers
- Common in deep CNNs and RNNs
- Severe with sigmoid and tanh activations

Mathematical Explanation

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial a_n} \prod_{k=2}^n \frac{\partial a_k}{\partial a_{k-1}} \frac{\partial a_1}{\partial W_1} \quad (1)$$

- Chain rule multiplies many derivatives
- If derivatives < 1 , product $\rightarrow 0$
- Gradients vanish for earlier layers
- Saturating activation functions (Sigmoid, Tanh)
- Very deep architectures
- Poor weight initialization
- Long sequences in RNNs

Role of Activation Functions

- **Sigmoid**: derivative ≤ 0.25
- **Tanh**: derivative ≤ 1 but saturates
- **ReLU**: derivative $= 1$ (positive region)

Key Insight

Non-saturating activations help preserve gradient magnitude.

Effects on Training

Early layers stop learning, Slow convergence, Poor feature extraction, and Deeper networks perform worse than shallow ones

Solutions

- ReLU / Leaky ReLU / GELU
- Xavier and He initialization (for Relu)
- Batch Normalization / Layer Normalization
- Residual connections (ResNet)

Thank You